대용량 엑셀 다운로드 기능 구현 및 성능 검증

> Apache POI 기반 엑셀 다운로드 기능을 구현하고,
> JVM heap 조건별로 대용량 처리 한계를 검증한 사례

소개

JSP / Spring 기반 시스템에서 **DB 데이터를 xlsx 파일로 내려받는 엑셀 다운로드 기능**을 구현하고, 이후 **대용량 데이터에서 어디까지 안정적으로 동작하는지 검증한 사례**를 정리한 문서다. 화면의 `엑셀다운로드` 버튼을 누르면 DB 데이터를 조회해 파일로 생성하고, 사용자가 즉시 다운로드할 수 있도록 응답하는 구조로 구현했다.

문제 정의

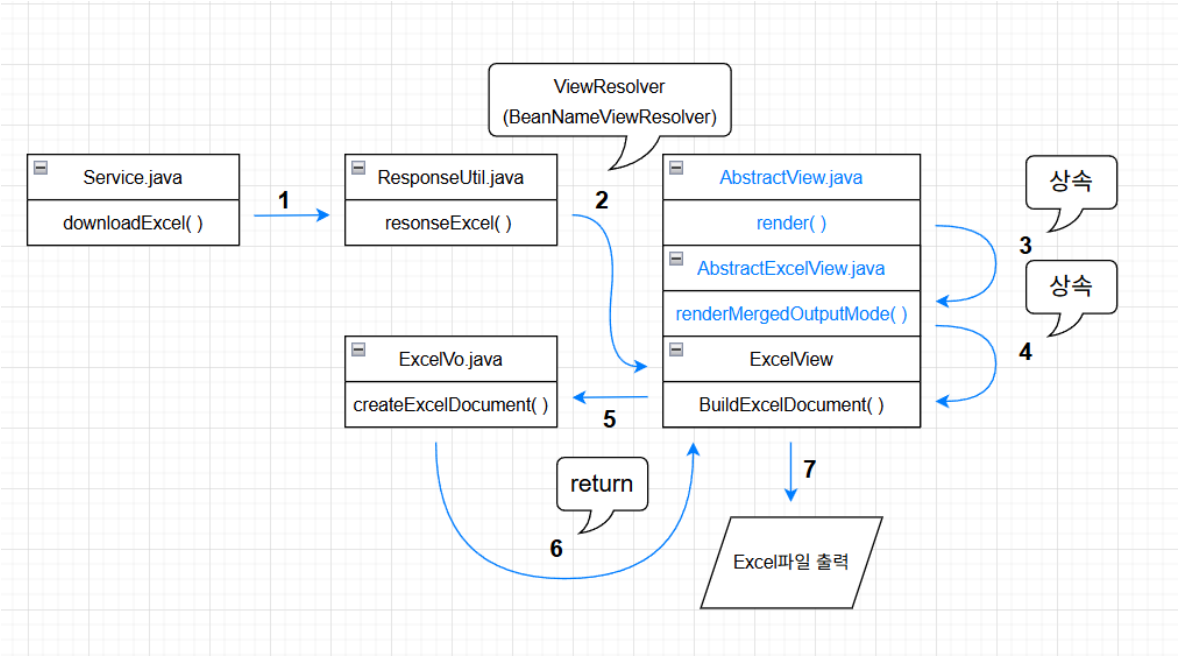
운영에서는 데이터 양이 증가할수록 **처리 시간**, **heap 사용량**, **오류 발생 시점**, **사용자에게 제공 가능한 최대 범위**를 함께 고려해야 한다. 이 기능은 Apache POI 로 구현되어 있었기 때문에, 행 수가 많아질수록 CPU와 메모리 사용량이 어떻게 증가하는지 검증할 필요가 있었다. 실험문서도 같은 문제의식에서 시작해, row 수에 따라 CPU와 RAM에 미치는 영향을 확인하는 자료로 작성되어 있다.

구현 구조

JSP → Service → ResponseUtil → ExcelView → ExcelVO → Download

JSP에서는 버튼 클릭 시 `downloadExcel()`을 호출한다. Service는 DB에서 조회한 데이터를 엑셀에 맞는 형식으로 가공하고, 헤더(`_headers`), 컬럼 키(`_items`), 실제 목록(`_list`)을 `ModelAndView`에 담는다. 이후 `ResponseUtil.responseExcel()`이 지정된 `excelView`를 호출하도록 연결하고, `ExcelView`는 파일명과 HTTP 헤더를 설정한 뒤 `ExcelVO`가 실제 시트와 셀 내용을 작성하도록 위임한다. 즉, **데이터 준비**, **View 연결**, **응답 설정**, **시트 작성**이 각각 나뉘어 있다.

컬럼 구성이나 데이터 매핑을 바꾸고 싶으면 Service와 `ExcelVO`를 보면 되고, 파일명 설정이나 응답 헤더를 수정하고 싶으면 `ExcelView`를 보면 된다. 또한 `ExcelView`는 `SXSSFWorkbook`을 사용해 Excel 2007 이상 버전의 대용량 파일을 쓰기 전용으로 처리하고, flush 단위를 지정해 출력 스트림으로 바로 파일을 전달하도록 구성되어 있다.



구현 포인트

1. 데이터와 렌더링 책임 분리

Service는 데이터 조회와 헤더/컬럼 구성을 담당하고, 실제 엑셀 시트 생성은 `ExcelVO`가 담당한다. 이 때문에 다운로드 로직이 한 메서드에 몰리지 않고, 출력과 생성이 분리되어 있다.

2. View 기반 다운로드 처리

`ResponseUtil.responseExcel()`은 `ModelAndView`에 대상 객체를 담고 `excelView`를 반환한다. 이후 Spring의 ViewResolver가 등록된 Bean을 찾아 `ExcelView`를 호출하고, 여기서 파일명 인코딩과 `Content-Disposition` 헤더 설정이 이뤄진다. 이 방식은 일반 화면 렌더링과 동일한 MVC 흐름 안에서 파일 다운로드를 처리한다는 장점이 있다.

3. 대용량 파일 대응 구조

`ExcelView`와 상위 View 계층은 `SXSSFWorkbook`을 사용하고, flush 행 단위를 설정하며, 출력 스트림에 직접 write / flush / close를 수행한다. 따라서 일반적인 규모의 엑셀뿐 아니라 **대용량 처리 가능성**을 염두에 둔 구조라고 볼 수 있다.

성능 검증 방법

구현 이후 어디까지 안정적으로 동작하는 지를 확인하기 위해 별도 실험을 진행했다. 실험은 Eclipse JRE 설정에서 `-Xms`, `-Xmx` 값을 변경하는 방식으로 수행했고, 각각의 조건에서 row 수를 늘려가며 **다운로드 성공 여부**, **소요 시간**, **CPU 최대 점유율**, **메모리 최대 점유율**, **스타일 적용 여부**를 기록했다. 실험 기준은 다음 세 가지였다.

- 기본 설정: `Xms256m -Xmx2048m`
- 확장 설정 1: `Xms1024m -Xmx8192m`
- 확장 설정 2: `Xms1024m -Xmx12288m`

결과

실험 조건	안정 구간	실패 시작 구간	대표 소요 시간	최대 메모리 점유	대표 이슈
Heap 2GB (`-Xmx2048m`)	약 **80,000행**	**90,000행~**	80,000행 기준 **30초**	약 **4.4GB**	`GC overhead limit exceeded`, `Java heap space`
Heap 8GB (`-Xmx8192m`)	약 **160,000행**	**170,000행~**	160,000행 기준 **2분**	약 **8.4GB**	heap 부족 시 동일 오류 발생
Heap 12GB (`-Xmx12288m`)	약 **250,000행**	**300,000행~**	250,000행 기준 **3분**	약 **12.5GB**	heap 부족 시 동일 오류 발생

실험 결과, JVM heap 크기에 따라 안정적으로 처리 가능한 최대 row 수가 뚜렷하게 달라졌다.

- **2GB heap**에서는 약 **80,000행**까지 안정적으로 다운로드 가능했고, 90,000행부터 `GC overhead limit exceeded` 또는 `Java heap space` 오류가 발생했다.
- **8GB heap**에서는 약 **160,000행**까지 안정적으로 다운로드 가능했고, 170,000행부터 오류가 발생했다. 이때 메모리 최대 점유율은 약 **8.4GB** 수준이었다.
- **12GB heap**에서는 약 **250,000행**까지 안정적으로 다운로드 가능했고, 300,000행부터 오류가 발생했다. 메모리 최대 점유율은 약 **12.5GB** 수준이었다.

즉, heap을 확장하면 처리 가능한 최대 row 수는 증가했지만, CPU와 메모리 사용량도 함께 크게 증가했다. 특히 실패 구간에서는 CPU 점유율이 100%에 도달했고, heap 부족으로 인해 Garbage Collector가 계속 호출되면서 CPU 점유율이 상승한 것을 확인할 수 있었다.

발생한 문제

기능 자체는 대용량까지 확장 가능했지만, 몇 가지 주의할 점도 확인했다.

첫째, **스타일 적용 문제**. 실험 결과 스타일은 약 **6,000행 이하**에서는 정상 적용됐지만, 그 이상에서는 데이터는 생성되더라도 엑셀 파일을 열 때 스타일 제거 경고가 발생했다. 즉, **다운로드 성공 여부와 별개로 스타일이 먼저 깨지는 구간**이 존재했다.

둘째, **결과가 선형적으로 도출되지 않음.** 기본 설정 실험에서 80,000행을 안정 구간으로 정리했지만, 81,000행 이상에서도 간헐적으로 성공한 경우가 존재한다. 다만 행 수가 증가할수록 실패 확률이 높아졌고, 전체적으로는 불안정성이 커지는 방향이었다.

셋째, **메모리 증설 ≠ 속도 향상.** 문서에서는 80,000행과 160,000행 비교 시 큰 시간 차이를 발견하지 못했으며, 시간 차이에 대한 자세한 결과는 컴퓨터 환경에 따른 실험이 필요하다고 보인다.

한계와 개선 방향

이 실험은 유의미한 기준을 제공하지만, 실제 운영 환경을 완전히 대변하지는 않는다. 실험에서 동시에 다른 프로그램을 실행하지 않은 단일 환경에서 측정했기 때문에, 운영 환경에서는 성공 가능한 행 수가 더 낮아질 수 있을 것으로 보인다.

또한 스타일 깨짐 이슈는 단순히 heap 크기만으로 설명되지 않는 별도의 품질 이슈가 존재한다. 향후에는 다음과 같은 개선 방향을 고려할 수 있다.

- 스타일 적용 방식 단순화 또는 분리
- row 수가 매우 큰 경우 CSV 등 대체 포맷 검토
- flush 정책과 workbook 설정 재조정
- 운영 환경과 유사한 부하 조건에서 반복 실험 수행

요약

엑셀 다운로드 기능의 구현 구조와 대용량 처리 한계를 함께 정리한 사례다.

구현 측면에서는 MVC 흐름 안에서 `Service → ResponseUtil → ExcelView → ExcelVO`로 책임을 나눠 다운로드를 처리했고, 검증 측면에서는 heap 크기별로 안정 구간과 오류 유형을 확인했다. 실험 결과 기준으로 안정 구간은 대략 **2GB heap = 80,000행**, **8GB heap = 160,000행**, **12GB heap = 250,000행** 수준이었다.